

1. Które stwierdzenie najlepiej opisuje różnicę między klasą abstrakcyjną a interfejsem w kontekście dziedziczenia w Java?
2. Klasa abstrakcyjna `Animal` posiada metodę abstrakcyjną `makeSound()` oraz metodę nieabstrakcyjną `sleep()`. Klasa `Dog` dziedziczy po `Animal` i implementuje `makeSound()`. Następnie klasa `GoldenRetriever` dziedziczy po `Dog`, ale nie nadpisuje `makeSound()`. Jakie jest zachowanie `GoldenRetriever`?
3. Który z poniższych modyfikatorów zastosowany do metody w klasie abstrakcyjnej spowoduje błąd kompilacji?
4. Co się stanie, gdy klasa abstrakcyjna nie zawiera żadnej metody abstrakcyjnej, ale posiada modyfikator `abstract` przy deklaracji klasy?
5. Klasa abstrakcyjna posiada konstruktor z parametrem. Klasa potomna nie definiuje żadnego konstruktora. Jakie jest zachowanie takiego kodu?
6. Które stwierdzenie dotyczące pól statycznych w klasie abstrakcyjnej jest prawdziwe?
7. Rozważmy hierarchię klas, gdzie klasa abstrakcyjna `Shape` posiada metodę abstrakcyjną `area()` oraz metodę nieabstrakcyjną `describe()`. Metoda `describe()` wewnętrznie wywołuje `area()`. Jak nazywa się ten wzorzec projektowy i jaka jest jego główna korzyść?
8. Co oznacza sytuacja, gdy klasa abstrakcyjna implementuje interfejs, ale nie implementuje wszystkich jego metod?
9. Dwie klasy abstrakcyjne `Vehicle` i `Machine` definiują metodę o tej samej nazwie `start()`. Klasa `Car` chce dziedziczyć po obu. Jakie jest ograniczenie Javy w tym kontekście i jak można je obejść?
10. Klasa abstrakcyjna posiada metodę z modyfikatorem `final` oraz metodę abstrakcyjną. Jakie jest zachowanie podklasy w stosunku do tych metod?
11. Klasa abstrakcyjna `AbstractProcessor` posiada chroniony konstruktor. Co to oznacza w praktyce dla kodu poza pakietem tej klasy?
12. Która z poniższych sytuacji spowoduje błąd w czasie kompilacji, a nie w czasie wykonania programu?
13. Jak zachowuje się mechanizm polimorfizmu, gdy referencja typu klasy abstrakcyjnej wskazuje na obiekt klasy konkretnej i wywoływana jest metoda, która istnieje zarówno w klasie abstrakcyjnej (jako nieabstrakcyjna), jak i jest nadpisana w klasie konkretnej?
14. Klasa abstrakcyjna `AbstractCache` definiuje metodę `get()` jako abstrakcyjną i metodę `getOrDefault()` jako nieabstrakcyjną, która wewnętrznie wywołuje `get()`. Programista tworzy anonimową klasę implementującą `AbstractCache` bezpośrednio w kodzie. Które stwierdzenie jest prawdziwe?

15. Programista tworzy hierarchię klas dla systemu płatności. Klasa abstrakcyjna `Payment` posiada metodę abstrakcyjną `validate()` i nieabstrakcyjną `process()`, która wywołuje `validate()` przed wykonaniem płatności. Klasa `CreditCardPayment` nadpisuje `validate()`. Następnie klasa `SecureCreditCardPayment` dziedziczy po `CreditCardPayment` i nadpisuje `validate()`. Gdy wywołamy `process()` na obiekcie `SecureCreditCardPayment`, która `validate()` zostanie wywołana?
16. Która z poniższych kombinacji modyfikatorów zastosowana do klasy jest poprawna składniowo w Javie?
17. W projekcie istnieje klasa abstrakcyjna `Repository` z metodą `findById()` zadeklarowaną jako abstrakcyjną. Dwie klasy - `UserRepository` i `ProductRepository` - dziedziczą po `Repository` i implementują `findById()`. Programista chce dodać wspólną logikę cachowania wyników `findById()` bez modyfikowania klas potomnych. Które podejście z użyciem klasy abstrakcyjnej jest najbardziej odpowiednie?
18. Klasa abstrakcyjna `AbstractTask` implementuje interfejs `Runnable` (z metodą `run()`) i oznacza `run()` jako abstrakcyjną. Klasa `ConcreteTask` dziedziczy po `AbstractTask` i implementuje `run()`. Programista przekazuje obiekt `ConcreteTask` do wątku `Thread`. Które stwierdzenie poprawnie opisuje tę sytuację?
19. Programista analizuje kod i stwierdza, że klasa abstrakcyjna ma 10 klas potomnych, z których 9 nadpisuje metodę `calculateFee()` identycznie, a jedna ma specyficzną implementację. Jakie jest najlepsze działanie refaktoryzacyjne z punktu widzenia zasad projektowania?
20. Która z poniższych cech klasy abstrakcyjnej w Javie jest prawdą w kontekście serializacji obiektów?
21. W klasie abstrakcyjnej zdefiniowano wyłącznie jeden, prywatny konstruktor. Jakie są konsekwencje architektoniczne takiej deklaracji dla tej klasy?
22. Próba dodania modyfikatora statycznego do metody abstrakcyjnej w Javie kończy się błędem kompilacji. Jaka jest główna teoretyczna przyczyna tego ograniczenia?
23. Klasa abstrakcyjna staje się częścią hierarchii klas podlegających serializacji, jednak sama nie implementuje interfejsu `Serializable`. Jakie zjawisko zajdzie podczas deserializacji obiektu klasy pochodnej?
24. Interfejs definiuje metodę domyślną (`default`). Klasa abstrakcyjna implementująca ten interfejs deklaruje tę samą metodę ponownie, ale z modyfikatorem `abstract`. Jaki będzie skutek takiego działania?
25. Projektant próbuje użyć modyfikatora `synchronized` w połączeniu z modyfikatorem `abstract` w deklaracji metody. Dlaczego kompilator Javy uzna to za poważny błąd semantyczny?
26. Wzorec projektowy `Template Method` często wykorzystuje klasy abstrakcyjne. Jakie kombinacje modyfikatorów dostępu i dziedziczenia są w nim najczęściej stosowane na poziomie metod?

27. W klasie zadeklarowano zagnieżdżoną (nested) klasę abstrakcyjną, która posiada modyfikator `private`. Jakie ograniczenia nakłada to na użycie tej klasy w kodzie aplikacji?
28. Programista deklaruje w klasie abstrakcyjnej metodę o sygnaturze `public abstract String toString()`. Jak wpływa to na zasady obiektowości i kompilację kodu?
29. Tworzona jest instancja przy użyciu klasy anonimowej, która rozszerza klasę abstrakcyjną posiadającą wyłącznie konstruktory z parametrami. Jak zachowa się kompilator?
30. W systemie opartym na Javie 17 zdefiniowano zapieczętowaną (sealed) klasę abstrakcyjną. Co wyróżnia to podejście w kontekście projektowania elastycznych hierarchii?
31. Zarówno klasa abstrakcyjna, jak i dziedzicząca po niej klasa konkretna zawierają niestatyczne bloki inicjalizacyjne. W jakiej kolejności zostaną one wykonane podczas tworzenia obiektu?
32. Klasa abstrakcyjna definiuje chronioną metodę statyczną. Klasa dziedzicząca definiuje metodę statyczną o identycznej sygnaturze. Jak specyfikacja Javy interpretuje tę sytuację?
33. Analizując różnice między nowoczesnymi interfejsami (Java 8+) a klasami abstrakcyjnymi, w jakim konkretnym przypadku użycie klasy abstrakcyjnej pozostaje technologicznie niezastąpione?
34. Metoda abstrakcyjna deklaruje zgłoszenie wyjątku kontrolowanego (checked exception) np. `IOException`. Jakie reguły obowiązują klasę podrzędną podczas nadpisywania tej metody?
35. Jak środowisko uruchomieniowe Javy interpretuje rzutowanie (casting) instancji istniejącego obiektu na zdefiniowany typ klasy abstrakcyjnej?
36. W konstruktorze klasy abstrakcyjnej programista wywołuje metodę, która jest zadeklarowana w tej klasie jako abstrakcyjna. Z jakim zjawiskiem wiąże się to działanie w praktyce?
37. Metoda abstrakcyjna otrzymała domyślny modyfikator dostępu (`package-private`). Klasa rozszerzająca tę klasę znajduje się w całkowicie innej przestrzeni pakietowej. Jakie będą tego skutki?
38. Ograniczenie wielokrotnego dziedziczenia klas (w tym abstrakcyjnych) ma kluczowe uzasadnienie w projektowaniu maszyny wirtualnej Javy. Które ze stwierdzeń najlepiej oddaje tę przyczynę architektoniczną?
39. Podczas nadpisywania metody abstrakcyjnej w klasie pochodnej, programista decyduje się na zmianę deklarowanego typu zwracanego. Jaka modyfikacja jest dopuszczalna przez kompilator?

40. Architekt decyduje się zadeklarować całą klasę z wykorzystaniem modyfikatora final. Później próbuje dodać do niej nowe metody oznaczone modyfikatorem abstract. Jaką ocenę wyda kompilator?
41. Które stwierdzenie najlepiej opisuje główny cel wprowadzenia rekordów w Javie (Java 16)?
42. Deweloper tworzy rekord i chce dodać własną logikę walidacji parametrów wejściowych. Które podejście jest zgodne ze specyfikacją rekordów w Javie?
43. Programista analizuje projekt i zastanawia się, czy rekord może rozszerzać inną klasę. Które stwierdzenie jest prawdziwe?
44. Zespół rozważa użycie rekordów jako kluczy w HashMap. Które stwierdzenie najlepiej uzasadnia, czy jest to bezpieczne podejście?
45. Deweloper chce dodać do rekordu dodatkowe pole instancji, które nie jest częścią jego komponentów. Jaki będzie wynik próby kompilacji takiego kodu?
46. Architekt systemu analizuje, w którym scenariuszu użycie rekordu byłoby nieodpowiednie. Które uzasadnienie jest merytorycznie poprawne?
47. Deweloper zastanawia się nad modyfikowalnością komponentów rekordu. Które stwierdzenie precyzyjnie opisuje gwarancje niemutowalności rekordu?
48. Programista porównuje rekordy z klasami oznaczonymi jako final zawierającymi tylko pola final. Która cecha odróżnia rekord od takiej klasy?
49. Zespół używa biblioteki do serializacji i deserializacji JSON. Deweloper twierdzi, że rekordy mogą powodować problemy z niektórymi bibliotekami serializacji. Które wyjaśnienie jest poprawne?
50. Deweloper analizuje dwa podejścia do tworzenia kopii rekordu z jedną zmienioną wartością. Dlaczego w Javie nie istnieje wbudowana metoda withXxx() dla rekordów (w odróżnieniu od np. Kotlin data classes)?
51. Wyobraź sobie sytuację, w której projektujesz domenę aplikacji z użyciem rekordów i potrzebujesz współdzielić część stanów pomiędzy wieloma różnymi strukturami danych. Jakie są rzeczywiste ograniczenia językowe w tym zakresie?
52. Podczas tworzenia instancji rekordu zachodzi potrzeba zwalidowania wprowadzanych danych wejściowych oraz ewentualnej normalizacji ich wartości bez zmiany pierwotnych typów. Które stwierdzenie o konstruktorze kompaktowym jest prawdziwe?
53. Otrzymujesz strumień danych pochodzący z zewnętrznego systemu, który musi zostać zdeserializowany do struktury rekordu. W jakim stopniu mechanizm deserializacji gwarantuje zachowanie niezmienników zdefiniowanych w kodzie?

54. Programista definiuje rekord lokalny wewnątrz ciała metody realizującej skomplikowany algorytm, aby tymczasowo grupować wyniki pośrednie. W jaki sposób ten lokalny rekord wchodzi w interakcję ze zmiennymi otoczenia?
55. Jeden z komponentów nowo utworzonego rekordu przechowuje standardową, wbudowaną w język listę elementów tekstowych. Jakie są rzeczywiste gwarancje niezmienności stanu dla całej instancji tak zdefiniowanego rekordu?
56. Zachodzi potrzeba zmodyfikowania domyślnego sposobu zwracania wartości przez akcesor rekordu. Programista decyduje się na ręczne nadpisanie automatycznie generowanej metody zwracającej wartość konkretnego komponentu.
57. Zespół tworzy uniwersalny framework do mapowania struktur obiektowych na format tekstowy i musi dynamicznie identyfikować struktury rekordowe oraz pobierać metadane o ich wewnętrznych komponentach w czasie działania programu.
58. Na deklaracji pojedynczego komponentu w nagłówku rekordu została umieszczona standardowa adnotacja walidacyjna wykorzystywana przez popularne narzędzia do mapowania. W jaki sposób kompilator zarządza rozmieszczeniem tej adnotacji?
59. Architektura aplikacji wymaga owinięcia instancji danych w obiekt pośredniczący, który będzie monitorował czas dostępu do informacji. Wybrana biblioteka realizuje to zadanie poprzez dynamiczne generowanie klas potomnych w czasie działania programu.
60. Instancja rekordu posiada komponent, którym jest referencja do standardowej, wieloelementowej tablicy liczbowej. Ten sam obiekt rekordu jest jednocześnie odczytywany i modyfikowany przez wiele niezależnych procesów wykonywanych równolegle.
61. Interfejs Printable definiuje metodę default `print()` z pewną implementacją. Interfejs Exportable również definiuje metodę default `print()` z inną implementacją. Klasa Document implementuje oba interfejsy, ale nie nadpisuje metody `print()`. Co się stanie podczas kompilacji?
62. Programista deklaruje pole w interfejsie bez żadnych modyfikatorów: `int MAX_SIZE = 100`. Jakie modyfikatory zostaną automatycznie zastosowane przez kompilator?
63. Interfejs Validator w Javie 9 definiuje metodę prywatną `validateFormat()` oraz metodę default `validate()`, która wywołuje `validateFormat()`. Klasa EmailValidator implementuje Validator. Które stwierdzenie jest prawdziwe dotyczące metody `validateFormat()`?
64. Interfejs Repository definiuje metodę static `createInstance()`. Klasa UserRepository implementuje Repository. Programista próbuje wywołać metodę przez `UserRepository.createInstance()`. Jaki będzie wynik?
65. Interfejs A definiuje metodę default `greet()` zwracającą "Hello from A". Interfejs B rozszerza A i nadpisuje `greet()` zwracając "Hello from B". Klasa C implementuje B bez nadpisywania `greet()`. Jaki wynik zwróci wywołanie `greet()` na obiekcie klasy C?

66. Programista chce zapewnić że każda klasa implementująca interfejs Shape będzie porównywalna z innymi obiektami Shape pod względem wielkości pola. Które podejście jest najbardziej idiomatyczne w Javie?
67. Które stwierdzenie poprawnie opisuje przewagę interfejsu znacznikowego nad adnotacją z retencją RUNTIME przy weryfikacji cech obiektów?
68. Interfejs EventListener posiada jedną metodę abstrakcyjną onEvent() oraz dwie metody default. Programista chce użyć wyrażenia lambda do przekazania implementacji. Co jest wymagane?
69. Klasa abstrakcyjna AbstractAnimal posiada nieabstrakcyjną metodę move(). Interfejs Swimmable również definiuje metodę default move(). Klasa Duck dziedziczy po AbstractAnimal i implementuje Swimmable bez nadpisywania move(). Która wersja move() zostanie wywołana?
70. Programista zauważa że interfejs ma 15 metod abstrakcyjnych, a większość klas implementujących dostarcza puste implementacje dla 10 z nich. Które rozwiązanie najlepiej adresuje ten problem zgodnie z zasadami SOLID?
71. Klasa BaseService implementuje interfejs Configurable z metodą default configure() i chce wywołać tę domyślną implementację wewnątrz własnej nadpisanej wersji configure(). Która składnia jest poprawna?
72. Które wymaganie jednoznacznie przemawia za wyborem klasy abstrakcyjnej zamiast interfejsu do modelowania komponentu systemu?
73. Interfejs C rozszerza interfejsy A i B, oba definiujące metodę default calculate(). Interfejs C nie nadpisuje calculate(). Klasa D implementuje C bez nadpisywania calculate(). Co się stanie podczas kompilacji?
74. Interfejs Cache definiuje metodę default put() implementującą logowanie, która wewnętrznie wywołuje abstrakcyjną metodę store(). Klasa implementująca dostarcza implementację store(). Jak nazywa się ten wzorzec i co jest jego główną korzyścią w kontekście interfejsów względem klas abstrakcyjnych?
75. Programista wydał bibliotekę z publicznym interfejsem API zawierającym 5 metod. Po roku chce dodać 6. metodę bez łamania istniejących implementacji klientów. Które podejście zachowuje kompatybilność binarną i źródłową?
76. Interfejs Processor jest interfejsem funkcyjnym z metodą process(T input). Programista chce dodać możliwość kompozycji dwóch procesorów tak by wynik pierwszego był wejściem drugiego. Które podejście zachowuje charakter funkcyjny interfejsu?
77. Klasa implementuje Comparable<T> i dostarcza metodę compareTo(). Programista zwraca z compareTo() wartości -1, 0 lub 1. Kolega twierdzi że to błąd i należy zwracać wynik odejmowania porównywanych wartości całkowitych. Kto ma rację i dlaczego?

78. Dwa interfejsy: Flyable z default move() zwracającym "flying" i Drivable z default move() zwracającym "driving". Klasa FlyingCar implementuje oba i nadpisuje move() wywołując Flyable.super.move() oraz Drivable.super.move() i łącząc wyniki. Co zwróci move()?
79. Programista analizuje projekt gdzie interfejsy zawierają rozbudowaną logikę biznesową w metodach default. Z perspektywy zasad SOLID które naruszenie jest najbardziej bezpośrednim skutkiem tego podejścia?
80. Interfejs AutoCloseable deklaruje close() rzucający Exception. Interfejs Closeable rozszerza AutoCloseable i deklaruje close() rzucający IOException. Klasa implementuje Closeable. Która deklaracja throws w metodzie close() tej klasy jest nie-
dozwolona?
81. Klasa implementuje dwa niezależne interfejsy, z których każdy definiuje metodę domyślną o identycznej sygnaturze. Jak kompilator Javy zareaguje na taką strukturę?
82. Od wersji Java 9 możliwe jest definiowanie prywatnych metod wewnątrz interfejsów. Jaki jest główny cel wprowadzenia tego rozwiązania architektonicznego?
83. Projektant próbuje zadeklarować w interfejsie metodę domyślną o sygnaturze identycznej z metodą z klasy Object. Jak zareaguje analizator składni kompilatora?
84. Programista deklaruje zmienną przypisując jej wartość bezpośrednio w ciele interfejsu. Jakie modyfikatory dostępności są do niej niejawnie przypisywane?
85. Interfejs definiuje jedną własną metodę abstrakcyjną oraz jawnie deklaruje metodę abstrakcyjną odpowiadającą sygnaturze metody equals. Czy jest to interfejs funkcyjny?
86. Zmienna przetrzymywana w referencji interfejsu pierwszego zostaje jawnie rzutowana na interfejs drugi, który nie jest z nim spokrewniony. Co się wydarzy podczas kompilacji?
87. Czy publiczna metoda statyczna zdefiniowana wewnątrz interfejsu może w swoim ciele bezpośrednio wywołać metodę domyślną tego samego interfejsu?
88. Modernizacja specyfikacji (Java 15+) wprowadziła mechanizm interfejsów zapieczętowanych (sealed). Jaki obowiązek nakładają one na implementujące je klasy?
89. Tworzona jest klasa anonimowa implementująca interfejs. W jej wnętrzu programista dodaje nową, własną metodę publiczną. Jaka będzie jej faktyczna użyteczność?
90. Metoda z interfejsu deklaruje zgłaszanie wyjątku IOException. Klasa ją implementująca deklaruje w nadpisanej metodzie rzucanie wyjątku Exception. Jaki będzie wynik?
91. Klasa abstrakcyjna implementuje interfejs zawierający metodę domyślną. Następnie klasa ta redefiniuje ową metodę używając słowa abstract. Jakie rodzi to skutki?

92. Czym podyktowany jest częsty wybór pustych interfejsów znacznikowych (np. Cloneable) zamiast mechanizmu adnotacji do flagowania klas w zaawansowanych systemach Javy?
93. Interfejsy strukturalnie różnią się od klas całkowitym zakazem deklarowania pól zmiennych instancyjnych. Jakie zjawisko architektoniczne powstrzymuje ten rygorystyczny zakaz?
94. Klasa implementuje dwa interfejsy posiadające metodę domyślną. Jak programista może jawnie odwołać się do implementacji pochodzącej bezpośrednio z wybranego interfejsu?
95. Zgodnie ze standardem maszyny wirtualnej Javy (JVM), w którym precyzyjnym momencie cyklu życia programu następuje ostateczna, statyczna inicjalizacja interfejsu?
96. Zdefiniowano interfejs udostępniający publiczną metodę statyczną. Zwykła klasa podjęła się implementacji tego interfejsu. Czy klasa ta może wywołać tę metodę przez własną nazwę?
97. Metoda w interfejsie deklaruje zwracanie obiektu typu Number. Klasa nadpisująca tę metodę modyfikuje kod, ustalając typ zwracany na Integer. Jaka będzie ocena tej operacji?
98. Początkujący deweloper próbuje utworzyć wewnątrz interfejsu abstrakcyjną metodę, poprzedzając ją modyfikatorem protected. Dlaczego próba ta kończy się szybkim niepowodzeniem?
99. Zdefiniowana w interfejsie publiczna metoda abstrakcyjna zostaje wdrożona w klasie. Programista decyduje się zmienić dostęp jej implementacji na chroniony. Czy kod skompiluje się poprawnie?
100. Architekt decyduje się przechowywać stałe wartości konfiguracyjne aplikacji prosto w polach interfejsu, odczytując je na przestrzeni wielu wątków. Jak rozwiązana jest kwestia współbieżności?
101. Które stwierdzenie najlepiej opisuje wyrażenie lambda w Javie?
102. Co to jest interfejs funkcyjny w kontekście wyrażen lambda?
103. Adnotacja @FunctionalInterface zostanie zaakceptowana przez kompilator w przypadku interfejsu, który posiada:
104. Które z poniższych stwierdzeń dotyczących zmiennych używanych wewnątrz wyrażen lambda jest prawdziwe?
105. Pojęcie "effectively final" w kontekście wyrażen lambda oznacza, że zmienna:
106. Jakie jest główne praktyczne uzasadnienie wymagania "effectively final" dla zmiennych lokalnych w lambdach?

107. Czym różni się referencja do metody (method reference) od równoważnego wyrażenia lambda?
108. Który z pakietów Javy zawiera standardowe interfejsy funkcyjne takie jak Predicate, Function, Consumer i Supplier?
109. Interfejs Predicate<T> w Javie reprezentuje funkcję, która:
110. Interfejs Function<T, R> posiada metodę compose oraz andThen. Jaka jest różnica między nimi?
111. Które stwierdzenie dotyczące zakresu widzialności (scope) wyrażeń lambda jest prawdziwe?
112. Co zwraca metoda abstrakcyjna interfejsu Supplier<T>?
113. Która z poniższych właściwości NIE jest cechą wyrażeń lambda w Javie?
114. Jak JVM implementuje wyrażenia lambda wewnętrznie (na poziomie kodu bajtowego)?
115. Interfejs BiFunction<T, U, R> różni się od Function<T, R> tym, że:
116. Które stwierdzenie dotyczące słowa kluczowego "this" wewnątrz lambdy jest poprawne?
117. Które z poniższych stwierdzeń dotyczących interfejsów UnaryOperator<T> i BinaryOperator<T> jest prawdziwe?
118. Jaką rolę pełni metoda andThen w interfejsie Consumer<T>?
119. Które stwierdzenie najlepiej opisuje różnicę między leniwym (lazy) a chętnym (eager) przetwarzaniem w kontekście strumieni i lambda?
120. Rozważając aspekty projektowania API, który argument najsilniej przemawia za stosowaniem interfejsów funkcyjnych jako parametrów metod zamiast konkretnych klas z jedną metodą?
121. W jaki sposób kompilator Javy determinuje docelowy typ wyrażenia lambda podczas analizy semantycznej?
122. Analizując cykl życia zmiennych, dlaczego wyrażenie lambda może korzystać ze zmiennych lokalnych otaczającej metody tylko wtedy, gdy są one "effectively final"?
123. Dokonując ewaluacji obsługi wyjątków, który z poniższych mechanizmów jest prawidłowy w przypadku wystąpienia wyjątku typu "checked" wewnątrz wyrażenia lambda przypisanego do interfejsu Consumer?
124. Porównując zachowanie referencji do metody w formacie "obiekt::metoda" oraz analogicznego wyrażenia lambda, jaka jest główna różnica w procesie ewaluacji obiektu docelowego?

125. Ewaluując potencjalne wycieki pamięci w aplikacjach Java, w jakiej sytuacji użycie wyrażenia lambda może spowodować utrzymanie obiektu w pamięci dłużej niż zakładano?
126. Analizując strukturę interfejsów, który z poniższych warunków musi zostać spełniony, aby dany interfejs mógł być uznany za interfejs funkcyjny w architekturze Java?
127. Ewaluując wydajność mechanizmów Javy, dlaczego użycie wyrażenia lambda jest bardziej optymalne w czasie uruchamiania niż tradycyjne anonimowe klasy wewnętrzne?
128. Analizując bezpieczeństwo wielowątkowej aplikacji, w jakim przypadku zastosowanie wyrażenia lambda stwarza potencjalne zagrożenie w środowisku współbieżnym?
129. Porównując procesy kompozycji funkcji z wykorzystaniem interfejsu Function, jaka jest kluczowa różnica architektoniczna pomiędzy metodami compose oraz andThen?
130. W procesie kompilacji kodu, jak kompilator rozwiązuje konflikt, gdy wyrażenie lambda jest przekazywane do przeciążonej metody, a struktura lambdy pasuje do dwóch różnych interfejsów funkcyjnych?
131. Analizując możliwości sieciowe, jakie restrykcje muszą być spełnione, aby instancja wyrażenia lambda mogła zostać poddana standardowemu procesowi serializacji w Javie?
132. Ewaluując hierarchię pakietu java.util.function, z jakiego powodu zdecydowano się na wprowadzenie odrębnego interfejsu BinaryOperator zamiast korzystania wyłącznie z ogólnego BiFunction?
133. Analizując semantykę języka, dlaczego wewnątrz wyrażenia lambda dozwolona jest modyfikacja pól instancyjnych klasy otaczającej, podczas gdy zmiana zmiennych lokalnych metody jest zabroniona?
134. Oceniając optymalizacje wydajności przy użyciu interfejsu Supplier, dlaczego zaleca się przekazywanie instancji Suppliera w bibliotekach logujących zamiast bezpośrednio konkatenowanych stringów?
135. Dokonując analizy pakietu java.util.function, jaki jest główny architektoniczny cel istnienia specjalizowanych interfejsów funkcyjnych takich jak IntPredicate czy DoubleFunction?
136. Dokonując ewaluacji różnic między klasami anonimowymi a lambda, na który obiekt wskazuje słowo kluczowe "this" użyte wewnątrz ciała wyrażenia lambda?
137. W kontekście projektowania własnych bibliotek, jakie konsekwencje niesie za sobą usunięcie adnotacji FunctionalInterface z nad interfejsu posiadającego jedną metodę abstrakcyjną?
138. Analizując strukturę klasycznych interfejsów w nowszych wersjach Javy, w jaki sposób obecność metod domyślnych w interfejsie wpływa na wyrażenia lambda?

139. Ewaluując zasady widoczności zakresu, co stanie się podczas weryfikacji syntaktycznej, jeżeli parametr wyrażenia lambda otrzyma tę samą nazwę, co zmienna lokalna zadeklarowana w metodzie przed deklaracją tejże lambda?
140. Zastanawiając się nad zastosowaniem referencji do konstruktora postaci `Klasa::new`, w jaki sposób mechanizm Javy decyduje, który konkretnie konstruktor obiektowy ma zostać faktycznie wywołany?
141. Które stwierdzenie dotyczące typów wylicznikowych (enum) w Javie jest NIEPOPRAWNE?
142. Jaką metodę należy użyć, aby uzyskać tablicę wszystkich wartości zdefiniowanych w danym typie wylicznikowym?
143. Która z poniższych właściwości dotyczących konstruktora w typie wylicznikowym jest prawdziwa?
144. Metoda `ordinal()` zdefiniowana w klasie `java.lang.Enum` zwraca:
145. Co się stanie, gdy wywołamy metodę `Enum.valueOf()` z nazwą, która nie odpowiada żadnej stałej w danym typie wylicznikowym?
146. Które stwierdzenie dotyczące użycia typów wylicznikowych w instrukcji `switch` jest poprawne?
147. Rozważasz projekt, w którym chcesz przypisać każdej stałej wylicznikowej dodatkowe dane (np. kod HTTP i opis). Które podejście jest architektonicznie poprawne w Javie?
148. Klasa `EnumSet`, dostępna w pakiecie `java.util`, różni się od zwykłego `HashSet` dla typów wylicznikowych przede wszystkim tym, że:
149. Jakie jest zachowanie metody `compareTo()` odziedziczonej przez typy wylicznikowe z klasy `java.lang.Enum`?
150. W kontekście serializacji, które stwierdzenie dotyczące typów wylicznikowych w Javie jest prawdziwe?
151. Analizując proces serializacji w języku Java, jak zachowa się maszyna wirtualna (JVM) podczas deserializacji typu wylicznikowego, który posiada dodatkowe, modyfikowalne pola instancyjne zmienione w trakcie działania programu przed jego zapisaniem?
152. Oceniając wady i zalety różnych implementacji wzorca projektowego Singleton w środowisku wielowątkowym, dlaczego wykorzystanie typu wylicznikowego uznaje się w Javie za podejście najbezpieczniejsze architektonicznie?
153. Wybierając optymalną strukturę danych do przechowywania zbioru elementów wylicznika w systemie o rygorystycznych wymaganiach wydajnościowych, decydujesz się na `EnumSet`. Jak wewnętrznie ewaluowana jest ta struktura dla wylicznika posiadającego 40 elementów?

154. Rozpatrz problem rozszerzalności typów. Student próbuje stworzyć typ wylicznikowy, który dziedziczy po klasie bazowej zawierającej wspólną logikę biznesową, jednak kompilator zgłasza błąd. Jaka fundamentalna zasada działania kompilatora Javy jest tego przyczyną?
155. Analizując dostępne API typów wylicznikowych, programista wywołuje statyczną metodę `values()` w celu iteracji po wszystkich elementach. Gdzie fizycznie znajduje się implementacja tej metody?
156. Dokonując ewaluacji jakości kodu (code review), zauważasz, że typ wylicznikowy posiada modyfikowalne pola stanu oraz standardowe metody typu setter. Jakie jest największe zagrożenie architektoniczne wynikające z takiego podejścia w aplikacji webowej?
157. Ewaluując zależności między modułami, zastanawiasz się, jak instrukcja `switch` wykorzystująca wyłącznie typ wylicznikowy zachowa się po rearanżacji kolejności elementów w kodzie źródłowym `enum`a, jeśli moduł zawierający instrukcję `switch` nie zostanie zrekompileowany. Jaki mechanizm chroni aplikację przed błędnym sterowaniem przepływem?
158. Tworząc architekturę parsera komend, w której każdy element typu wylicznikowego (reprezentujący komendę) implementuje własną wersję metody abstrakcyjnej zdefiniowanej wewnątrz tego `enum`a. W jaki sposób maszyna wirtualna Javy traktuje te indywidualne implementacje dla każdego z elementów?
159. Dokonując analizy profilowania pamięci serwera aplikacji, badasz cykl życia instancji typu wylicznikowego. W jakich okolicznościach odśmiecaz pamięci (Garbage Collector) usunie obiekty reprezentujące konkretne elementy wylicznika z pamięci sterty (Heap)?
160. Oceniając integralność danych w bazie relacyjnej, junior developer proponuje utrwalanie wartości typu wylicznikowego poprzez bezpośredni zapis wartości zwracanej przez metodę `ordinal()`. Jakie architektoniczne zagrożenie wiąże się z tą powszechną, lecz niebezpieczną praktyką?
161. Które stwierdzenie najlepiej opisuje zjawisko `type erasure` w Javie?
162. Jaka jest różnica między `List<? extends Number>` a `List<? super Number>`?
163. Co się stanie podczas próby wykonania następującej operacji w czasie wykonania programu: sprawdzenie czy obiekt jest instancją `List<String>` za pomocą operatora `instanceof`?
164. Która z poniższych deklaracji metody generycznej jest niepoprawna składniowo?
165. Czym jest `raw type` i jakie jest jego główne ryzyko?
166. Które z poniższych stwierdzeń dotyczących kowariancji tablic w Javie jest prawdziwe i najlepiej wyjaśnia, dlaczego typy generyczne nie są kowariantne?
167. Co oznacza zapis `<T extends Comparable<? super T>`?

168. Który z poniższych fragmentów kodu spowoduje błąd kompilacji w związku z użyciem typów generycznych?
169. Jak zachowuje się bounded wildcard `<? extends T>` w kontekście zapisu do kolekcji?
170. Co oznacza pojęcie heap pollution w kontekście typów generycznych?
171. Dlaczego nie można tworzyć tablic typów generycznych, np. `new T[10]` wewnątrz klasy generycznej?
172. Która z poniższych cech odróżnia metodę generyczną od klasy generycznej?
173. Co to jest wildcard capture i w jakim kontekście jest stosowany?
174. Jakie jest zachowanie kompilatora przy wywołaniu metody z varargs typów generycznych (np. `@SafeVarargs`)?
175. Które stwierdzenie dotyczące dziedziczenia i typów generycznych jest poprawne?
176. Czym różni się `<T extends A & B>` od `<T extends A>` z osobnym ograniczeniem `<U extends B>`?
177. Co to jest super type token i do czego służy?
178. Jakie jest zachowanie metody `Collections.sort(List<T> list)` w kontekście ograniczeń typowych, jeśli `T` nie implementuje `Comparable`?
179. Która z poniższych sytuacji poprawnie opisuje przypadek, w którym użycie `raw type` może prowadzić do błędu w czasie wykonania, mimo braku ostrzeżeń kompilatora?
180. Analizując mechanizm wymazywania typów (type erasure) w Javie, jak kompilator traktuje nieograniczone parametry typu podczas generowania kodu bajtowego?
181. W kontekście reguły PECS (Producer Extends, Consumer Super), oceniając projekt biblioteki wymagającej metody dodającej nowe elementy do przekazanej kolekcji, jaki typ parametru należy zastosować?
182. Dlaczego język Java kategorycznie zabrania bezpośredniego tworzenia instancji tablic generycznych w pamięci sterty?
183. Analizując zjawisko “zanieczyszczenia sterty” (heap pollution), w jakiej specyficznej sytuacji kompilator zgłosi odpowiednie ostrzeżenie?
184. Jaką główną rolę pełnią metody pomostowe (bridge methods) generowane automatycznie przez kompilator?
185. Dokonując ewaluacji różnic, jaka jest najistotniejsza różnica między użyciem typu surowego (raw type) a kolekcji z nieograniczonym znakiem wieloznacznym?
186. Projektując klasę, definiujemy parametr generyczny posiadający wiele ograniczeń górnych (wielokrotne wiązania). Jaka zasada bezwzględnie obowiązuje przy takiej deklaracji?

187. Z jakiego powodu pola statyczne w klasie generycznej nie mogą używać parametrów typu zadeklarowanych na poziomie tej klasy?
188. Analizując ograniczenia systemu wyjątków, dlaczego niemożliwe jest stworzenie generycznej klasy dziedziczącej po głównej klasie błędów systemowych?
189. W jaki sposób interfejs refleksji systemowej odnosi się do problemu wymazywania typów generycznych w Javie?
190. Dokonując oceny sygnatur metod, dlaczego kompilator odrzuci próbę przeciążenia (overloading) dwóch metod różniących się wyłącznie typem argumentów generycznych w przekazywanej kolekcji?
191. Czym charakteryzuje się zjawisko “przechwytywania znaków wieloznacznych” (wild-card capture) podczas kompilacji?
192. Zastosowanie dedykowanej adnotacji, której zadaniem jest tłumienie ostrzeżeń dotyczących zanieczyszczenia sterty przy zmiennej liczbie argumentów generycznych, jest obwarowane restrykcjami. Na jakich metodach wolno ją stosować?
193. Analizując rekursywne wiązania typów (recursive type bounds), jaki główny cel przyświeca projektowaniu struktur używających parametru generycznego ograniczonego przez samego siebie?
194. Dlaczego powszechnie uznawaną dobrą praktyką architektoniczną jest unikanie zwracania z metod typów opartych na ograniczonych znakach wieloznacznych?
195. Ewaluując inwariantność standardowych kolekcji generycznych: dlaczego lista tekstów nie jest traktowana przez kompilator jako podtyp listy głównego typu obiektowego?
196. Czy w standardowej klasie pozbawionej parametrów generycznych można zdefiniować metodę, która posiada własne, niezależne parametry generyczne?
197. Wyobraźmy sobie interfejs funkcyjny, który potrafi zarówno przyjmować dane określonego typu, jak i je modyfikować (zwracać ten sam typ). Jaką wariację reprezentuje jego parametr generyczny względem reguł elastyczności typów?
198. Dokonując analizy działania operatora diamentowego (diamond operator) dodanego w późniejszych wersjach języka: w jakiej specyficznej sytuacji jego mechanizm wnioskowania o typie okaże się bezsilny i wymusi określenie typu?
199. Dlaczego powszechnie stosowaną praktyką jest rzutowanie wartości na typ o przecięciowym charakterze (intersection type) poprzez spajanie operatorami przy wykorzystaniu rzutowania z generykami w niektórych bibliotekach?
200. Która z poniższych implementacji interfejsu Map w Javie gwarantuje zachowanie kolejności wstawiania elementów?
201. Jaki jest rzeczywisty efekt wywołania metody `iterator.remove()` na obiekcie `Iterator` pobranym z kolekcji `ArrayList` podczas iteracji po tej kolekcji?

202. Co różni interfejsy Comparable i Comparator w kontekście sortowania kolekcji w Javie?
203. Który z poniższych zestawów cech najdokładniej opisuje klasę PriorityQueue z pakietu java.util?
204. Dlaczego użycie mutowalnego obiektu jako klucza w HashMap jest uznawane za błąd projektowy?
205. Jaką złożoność czasową mają operacje add, remove i contains dla struktury danych HashSet w przypadku pesymistycznym?
206. W jaki sposób metoda Collections.unmodifiableList() chroni oryginalną kolekcję przed modyfikacją?
207. Co jest główną różnicą między ArrayDeque a LinkedList, gdy obie są używane jako implementacja interfejsu Deque?
208. Który z poniższych scenariuszy poprawnie wyjaśnia, kiedy należy preferować TreeSet zamiast HashSet?
209. Jakie jest zachowanie metody putIfAbsent() w HashMap w porównaniu do standardowej metody put()?
210. Weryfikacja mechanizmu optymalizacji kolizji w HashMapie (od Javy 8). Zewnętrzny system generuje klucze o bardzo słabej dystrybucji skrótów. W jakich dokładnie warunkach wewnętrzna struktura pojedynczego kubła (bucket) zmieni się z listy wiązanej w zbalansowane drzewo czerwono-czarne?
211. Architekt nowoczesnego systemu rekomendacyjnego narzucił użycie ArrayList zamiast LinkedList do przechowywania milionów obiektów, pomimo faktu, że system często wstawia nowe elementy w środku kolekcji. Jaka cecha wewnętrzna architektury sprzętowej i wirtualnej maszyny uzasadnia tę z pozoru nielogiczną decyzję?
212. Deweloper umieścił niestandardowy obiekt w kolekcji HashSet, a następnie za pomocą metody ustawiającej (setter) zmienił wartość pola, które brało czynny udział w obliczaniu skrótu (hash) dla tego obiektu. Jak zachowa się struktura podczas próby sprawdzenia obecności tego obiektu?
213. Analizujesz wąskie gardło w aplikacji wielowątkowej. Zauważasz, że w jednym z komponentów masowo wykorzystywana jest kolekcja CopyOnWriteArrayList. W jakim specyficznym scenariuszu użycie tej właśnie implementacji przyniesie mierzalne korzyści wydajnościowe w stosunku do standardowej synchronizacji?
214. Dokonujesz przeglądu kodu i zauważasz użycie IdentityHashMap zamiast standardowej HashMapy. W jakich przypadkach logika działania tej specyficznej struktury wpływa na poprawność działania programu w porównaniu do standardowych implementacji?

215. Wyjątek `ConcurrentModificationException` często zaskakuje początkujących programistów, szczególnie w aplikacjach jednowątkowych. Które z poniższych działań stanowi ścisłą techniczną przyczynę wyrzucenia tego błędu przez mechanizm obronny iteratora kolekcji `ArrayList`?
216. Projektujesz klasę wstawianą do struktury `TreeSet`. Przesłaniasz metodę porównującą tak, aby zwracała zero, gdy dwa obiekty mają ten sam identyfikator, jednak zapominasz zaimplementować metodę odpowiadającą za ich logiczną równość. Co się wydarzy przy próbie wstawienia dwóch takich obiektów?
217. Narzędzie do monitorowania pamięci wykazało użycie `WeakHashMap` w module zarządzania sesjami. Które ze zjawisk precyzyjnie opisuje przewagę i celowość stosowania tej struktury pod kątem mechanizmu odzyskiwania wolnej przestrzeni (`Garbage Collector`)?
218. Korzystasz z metody zewnętrznej biblioteki, która według dokumentacji zwraca widok kolekcji opakowany w `unmodifiableList`. Ewaluujesz z punktu widzenia bezpieczeństwa systemu, w jakim stopniu dane zwrócone przez ten interfejs są faktycznie zabezpieczone przed niepożądaną ingerencją?
219. Podczas optymalizacji algorytmu wyszukiwania ścieżki, użyłeś `PriorityQueue` w celu wyciągania elementów o najwyższym priorytecie. Jednak gdy sprawdzasz zawartość całej kolejki za pomocą domyślnego iteratora w pętli, zauważasz, że wyświetlane dane nie są poprawnie posortowane. Jaka jest tego przyczyna?
220. Która z poniższych cech najlepiej opisuje strumienie (`Stream`) w Javie w kontekście przetwarzania danych?
221. Jaka jest różnica między operacją pośrednią (`intermediate`) a terminalną (`terminal`) w `Stream API`?
222. Które stwierdzenie dotyczące metody `flatMap()` jest poprawne w porównaniu z `map()`?
223. Które z poniższych twierdzeń o metodzie `reduce()` jest fałszywe?
224. Jak działa kolektor `Collectors.groupingBy()` w `Stream API`?
225. Która metoda tworzenia strumienia będzie wymagała wywołania operacji terminalnej, aby przetworzyć elementy ze źródła danych strumienia?
226. Co zwróci wywołanie metody `count()` na strumieniu po użyciu `distinct()`?
227. Które stwierdzenie dotyczące równoległych strumieni (`parallel streams`) jest poprawne?
228. Jaka jest różnica między `findFirst()` a `findAny()` w `Stream API`?
229. Co się stanie gdy wywołamy operację terminalną na strumieniu, który został już wyczerpany?

230. Który kolektor z `Collectors` powinno się użyć, aby połączyć elementy strumienia `String` w jeden `String` z separatorem?
231. Jaka jest główna różnica między `Collectors.toUnmodifiableList()` a `Collectors.toList()`?
232. Co oznacza "short-circuit evaluation" w kontekście `Stream API`?
233. Które stwierdzenie o interfejsie `Collector` jest poprawne?
234. Jak zachowuje się metoda `peek()` w `Stream API`?
235. Która z metod `Stream` NIE jest operacją pośrednią?
236. Co rozróżnia `Collectors.partitioningBy()` od `Collectors.groupingBy()`?
237. Które stwierdzenie o strumieniach prymitywnych (`IntStream`, `LongStream`, `DoubleStream`) jest poprawne?
238. Jaka jest rola metody `Collectors.toMap()` i co się stanie gdy dwa elementy zmapują się na ten sam klucz?
239. Które stwierdzenie dotyczące `Optional` w kontekście `Stream API` jest najbardziej precyzyjne?
240. Kiedy projektujemy ciąg przetwarzania danych z wykorzystaniem wielu operacji pośrednich, w jaki sposób środowisko uruchomieniowe zarządza ich wykonaniem w kontekście leniwej ewaluacji?
241. Podczas przetwarzania strumienia w trybie równoległym programista decyduje się na dodawanie elementów bezpośrednio do zewnętrznej, standardowej listy zdefiniowanej wcześniej. Jaki będzie główny skutek takiego podejścia?
242. W systemie wykorzystano generator tworzący nieskończony strumień danych wejściowych. W jaki sposób należy go prawidłowo przetworzyć, aby uniknąć zawieszenia się całej aplikacji?
243. Analizujesz kod, w którym należy pobrać dziesięć pierwszych elementów spełniających określony, trudny do obliczenia warunek z ogromnego zbioru danych. Jak kolejność nakładania limitu i filtrowania wpływa na proces?
244. Operacje transformacji elementów w strumieniu projektuje się zazwyczaj jako funkcje czyste, pozbawione efektów ubocznych. Dlaczego używanie transformacji do modyfikowania stanu globalnego aplikacji jest błędem architektonicznym?
245. Wykonano operację agregującą na strumieniu użytkowników, która zwróciła ich sumę. Następnie w kolejnej linii kodu, korzystając z tej samej referencji do strumienia, podjęto próbę jego ponownego przefiltrowania. Jaki będzie tego efekt?
246. W procesie sprowadzania strumienia liczbowego do pojedynczej wartości (redukcji) wykorzystywany jest parametr tożsamości. Jaka pełni on rolę z punktu widzenia projektowania niezawodnej operacji?

247. Podczas integracji danych z systemów zewnętrznych otrzymujesz obiekty, z których każdy zawiera kolekcję sub-elementów. Chcesz otrzymać płaską strukturę wszystkich sub-elementów. Czym pod kątem działania różni się tutaj mapowanie płaskie od standardowego?
248. Zidentyfikowano nietypowy błąd występujący w środku złożonych operacji strumieniowych. Chcesz zarejestrować w logach bieżący stan przetwarzanych elementów, ale nie możesz ingerować w zwracany wynik. Jakiej operacji najlepiej użyć?
249. Decydujesz się na kategoryzację rozbudowanych struktur danych za pomocą standardowego kolektora grupującego. Jaki będzie techniczny rezultat wywołania tej wbudowanej funkcji zbierającej?
250. W aplikacji przetwarzającej transakcje uruchomiono wielowatkowe wyszukiwanie konkretnego warunku. Pod kątem mechanizmów wewnętrznych, dlaczego zadanie wyszukania absolutnie pierwszego elementu jest mniej optymalne od znalezienia jakiegokolwiek pasującego?
251. Przetwarzasz miliony odczytów z czujników wyrażonych w postaci liczb całkowitych. Jako inżynier optymalizujący zużycie RAMu decydujesz się na dedykowane strumienie dla typów prostych zamiast klasycznych strumieni obiektowych. Co uzyskasz dzięki tej zmianie?
252. Zasilasz mechanizm strumieniowy wykorzystując standardową, nieuporządkowaną kolekcję w której elementy zapewniają unikalność (zbiór). W jaki sposób wpłynie to na zachowanie operacji zależnych od położenia w sekwencji, takich jak ograniczanie wielkości strumienia?
253. Podczas projektowania modułu wyciągającego skrajne parametry narzędzi statystycznych, wykorzystujesz wbudowane funkcje wyluskujące wartość maksymalną. Dlaczego te metody nie zwracają od razu szukanego typu danych, lecz wykorzystują specjalną klasę zamykającą wynik (opakowanie)?
254. Analizując zużycie zasobów serwera zauważasz drastyczne skoki w użyciu pamięci podczas wykonywania niektórych operacji pośrednich, takich jak sortowanie całego zbioru czy usuwanie duplikatów. Jaka kategoria operacji powoduje ten problem?
255. Wykorzystujesz mechanizm kolektora do zbudowania mapy powiązań słownikowych pomiędzy ID a obiektem. Wskutek błędów w danych źródłowych, algorytm klasyfikujący napotyka drugi, identyczny klucz w ramach obecnego potoku przetwarzania. Co wydarzy się jeśli użyto standardowego, podstawowego kolektora dla map bez dodatkowych argumentów?
256. Zastanawiasz się nad sposobem odcięcia nagłówków z posortowanego chronologicznie dziennika zdarzeń. Rozważasz użycie klasycznego filtra bądź operacji odrzucającej warunkowo. Jaka jest kluczowa przewaga operacji odrzucającej warunkowo nad klasycznym filtrem dla tego scenariusza?

257. Implementujesz mechanizm autoryzacji sprawdzający, czy wszystkie role w liście roszczeń użytkownika spełniają wymogi systemowe używając w tym celu wbudowanej funkcji "czy wszystkie pasują". W wyniku błędu architektonicznego strumień ról użytkownika okazuje się zupełnie pusty. Jak zareaguje ewaluator w takiej skrajnej sytuacji?
258. Budujesz narzędzie agregujące tysiące mniejszych zbiorów tekstowych pochodzących od niezależnych klientów. Używasz petli, która w każdej iteracji wywołuje wbudowaną funkcję statycznego łączenia strumieni na bieżącym i nowym pliku. Jakie ryzyko operacyjne ukrywa ta iteracyjna technika?
259. W rozproszonej architekturze systemu odpalono kilkanaście równoległych strumieni do skomplikowanej analizy lokalnych zasobów, lecz jeden z nich niespodziewanie wywołuje w lambda-wyrażeniu powolne zapytania sieciowe REST ulegające częstemu opóźnieniu. Jaki systemowy efekt osiągnie ten wątek wobec całej maszyny JVM?
260. Który z poniższych mechanizmów w Javie zapewnia, że tylko jeden wątek może wykonywać dany blok kodu w tym samym czasie?
261. W jakim stanie musi znajdować się wątek, aby można było na nim wywołać metodę `notify()`?
262. Co się stanie, gdy wątek wywoła metodę `wait()` bez posiadania blokady na danym obiekcie?
263. Jaka jest różnica między `notify()` a `notifyAll()`?
264. Dlaczego wywołanie `wait()` powinno być umieszczone w petli `while`, a nie w instrukcji `if`?
265. Które stwierdzenie dotyczące bloku `synchronized` jest fałszywe?
266. Który z wymienionych scenariuszy NAJLEPIEJ opisuje problem "wygłodzenia" (starvation) wątków?
267. Co oznacza, że obiekt w Javie posiada "monitor"?
268. Jaka jest różnica między zsynchronizowaną metodą instancji a zsynchronizowanym blokiem na obiekcie `this`?
269. Wątek A posiada blokadę na obiekcie X i czeka na blokadę na obiekcie Y. Wątek B posiada blokadę na obiekcie Y i czeka na blokadę na obiekcie X. Jak nazywa się ta sytuacja i jak można jej zapobiec?
270. Które stwierdzenie poprawnie opisuje zachowanie wątku po wywołaniu metody `wait()`?
271. W kontekście synchronizacji, co to jest "sekcja krytyczna"?
272. Metoda `notify()` zostaje wywołana, ale żadne wątki nie są aktualnie oczekujące na danym monitorze. Co się dzieje?

273. Ktore podejscie do synchronizacji jest bardziej podatne na deadlock i dlaczego?
274. Ktore stwierdzenie o statycznych metodach synchronized jest prawdziwe?
275. Watek wywolal wait(5000) na pewnym obiekcie. Po jakim czasie watek moze zostac przebudzony?
276. Jaki jest glowny problem z uzywaniem notify() zamiast notifyAll() w sytuacji, gdy wiele watkow czeka na rozne warunki na tym samym monitorze?
277. Co to jest "race condition" w kontekście programowania wielowatkowego?
278. Ktore z ponizszych stwierdzen poprawnie opisuje relacje miedzy monitorem obiektu a metodami wait(), notify() i notifyAll()?
279. Programista chce zapewnić, że pewna operacja na wspoldzielonej liscie jest atomowa. Uzył synchronized na metodzie. Inny programista dodal do tej samej listy dostep z metody statycznej, rowniez synchronized. Czy ta synchronizacja jest wystarczajaca?
280. Rozważmy sytuację, w której wątek będący wewnątrz bloku zsynchronizowanego wywołuje inną metodę zsynchronizowaną na tym samym obiekcie. Jaki będzie rezultat takiej operacji na poziomie działania monitora?
281. Analizując mechanizmy czasowego wstrzymywania wątków, jaka jest fundamentalna różnica w zarządzaniu monitorem między wywołaniem metody usypiającej przypisanej do wątku, a metody oczekującej przypisanej do obiektu?
282. Co się stanie, gdy programista spróbuje zoptymalizować kod i wywoła metodę oczekującą na obiekcie poza jakimkolwiek blokiem lub metodą zsynchronizowaną przypisaną do tego obiektu?
283. W profesjonalnych implementacjach wzorców współbieżnych, wywołanie metody oczekującej na powiadomienie zawsze umieszcza się wewnątrz pętli warunkowej. Jaki architektoniczny problem rozwiązuje to konkretne podejście?
284. Oceniając użycie metod powiadamiających, w jakiej precyzyjnej sytuacji użycie powiadomienia pojedynczego wątku jest uznawane za w pełni bezpieczne i nie prowadzi do zablokowania całego systemu?
285. Wyobraźmy sobie scenariusz: Wątek pierwszy zajmuje zamek (monitor) obiektu X, a następnie próbuje zająć zamek obiektu Y. Jednocześnie wątek drugi zajmuje zamek Y, a następnie próbuje zająć X. Jaki będzie ostateczny wynik takiej architektury?
286. Analizując mechanizm zwalniania zamków (monitorów), co stanie się z monitorem chroniącym blok kodu, jeśli wewnątrz tego bloku zostanie rzucony nieobsłużony wyjątek czasu wykonania, który przerwie działanie wątku?
287. Aplikacja posiada metodę statyczną oraz metodę instancyjną, obie oznaczone słowem kluczowym oznaczającym synchronizację. Ewaluuując dostęp do tych metod z poziomu dwóch różnych wątków na tym samym obiekcie, jaki będzie wynik?

288. Dokonując inspekcji kodu natrafiasz na blok synchronizujący wykonanie na stałej tekstowej zapisanej bezpośrednio w kodzie. Dlaczego takie rozwiązanie architektoniczne jest uznawane za krytyczny błąd projektowy?
289. Analizując cykl życia wątku powiązanego z monitorem, w jakim precyzyjnie stanie znajduje się wątek bezpośrednio po odebraniu sygnału powiadamiającego, ale jeszcze przed faktycznym wznowieniem wykonywania kodu?
290. Zastanów się nad scenariuszem wyścigu: wątek pierwszy wywołuje metodę powiadamiającą monitor, zanim wątek drugi zdąży wywołać metodę oczekującą na tym samym obiekcie. Jaki jest rezultat przestrzenny takiej asynchronii?
291. Ewaluując cechy jakościowe wbudowanych mechanizmów synchronizacyjnych w języku Java, jak wygląda kwestia sprawiedliwości przydzielania dostępu do monitora w warunkach dużej rywalizacji wielu wątków?
292. Rozpatrz sytuację z punktu widzenia projektowania obsługi przerwań. Co dokładnie zachodzi, gdy wątek znajdujący się w stanie oczekiwania wewnątrz monitora otrzyma zewnętrzny sygnał przerywający jego działanie?
293. Dokonując analizy metody oczekującej z parametrem określającym maksymalny czas uśpienia, co musi się wydarzyć, aby wątek wznowił wykonywanie kodu tuż po upływie zdefiniowanego bezpiecznego limitu czasowego?
294. Zagłódzenie wątku to poważny błąd architektoniczny w systemach współbieżnych. Analizując mechanizmy wewnętrzne maszyny wirtualnej, która z poniższych sytuacji najtrafniej opisuje kontekst powstawania tego zjawiska w obrębie monitorów?
295. Badając zależności pomiędzy strukturą obiektową a systemem kontroli przepływu powiązanych z monitorami, o jakiej nadrzędnej zasadzie adresowania obiektów musisz pamiętać wywołując metodę oczekującą na powiadomienie?
296. Projektując mechanizm cyklicznych zadań współbieżnych programista postanowił ponownie użyć tego samego obiektu wątku, wywołując na nim metodę startową po raz drugi, po uprzednim zakończeniu jego pierwotnej pracy. Jak zachowa się środowisko?
297. Ewaluując warstwę abstrakcji biblioteki standardowej, w jaki sposób pod spodem zaimplementowana jest popularna metoda dołączania do wątku, która zatrzymuje proces wywołujący do momentu zakończenia procesu obserwowanego?
298. Porównując mechanizmy sterowania czasem procesora dla różnych wątków roboczych, w jaki sposób funkcja ustępowania czasu odnosi się bezpośrednio do logiki zarządzania wbudowanymi monitorami synchronizacji?
299. Analizując granulację zamków (monitorów) w obszarze pojedynczego obiektu klasy: co wynika z faktu synchronizowania wszystkich dostępnych modyfikatorów różnych i niezależnych logicznie zmiennych za pomocą słowa kluczowego na deklaracjach metod?